1) Create a simple login form for a mock Zomato admin panel that asks for username and password, then displays 'Access Granted' only if both fields match hardcoded admin credentials.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Zomato Admin Login</title>
<style>
body {
font-family: Arial, sans-serif;
background: #f5f5f5;
display: flex;
justify-content: center;
align-items: center;
height: 100vh;
margin: 0;
}
.login-box {
background: white;
padding: 35px;
width: 320px;
border-radius: 12px;
box-shadow: 0 4px 15px #ccc;
text-align: center;
}
h2 {
color: #e23744;
}
input {
width: 100%;
padding: 12px;
margin: 10px 0;
```

```
            box-sizing: border-box;
            border: 1px solid #ddd;
            border-radius: 6px;
        }
        button {
            width: 100%;
            padding: 12px;
            background: #e23744;
            color: white;
            border: none;
            border-radius: 6px;
            cursor: pointer;
            font-size: 16px;
        }
        button:hover {
            background: #c91e2b;
        }
        #message {
            margin-top: 15px;
            font-weight: bold;
        }
        </style>
        </head>
        <body>
        <div class="login-box">
<h2>ZOMATO</h2>
<h3>Admin Panel Login</h3>
        <form onsubmit="login(event)">
         <input type="text" id="username” placeholder="Enter Username" required>
         <input type="password" id="password" placeholder="Enter Password" required>
         <button type="submit">Login</button>
        </form>
```

```
    <p id="message"></p>
</div>
<script>
function login(event) {
    event.preventDefault();
    // Hardcoded admin credentials
    const adminUsername = "admin";
    const adminPassword = "admin123";
    const username = document.getElementById("username").value;
    const password = document.getElementById("password").value;
    const message = document.getElementById("message");
    if (username === adminUsername &&
        password === adminPassword) {
        message.style.color = "green";
        message.innerHTML = "Access Granted";
    } else {
        message.style.color = "red";
        message.innerHTML = "Invalid Username or Password";
    }
}
</script>
</body>
</html>
```

**2) Write a Python function check access (user role, resource) that returns True if the user role is allowed to access the resource, otherwise False. Test it with roles like 'customer', 'restaurant owner', and resources like 'menu_edit', 'order history', 'admin_dashboard'.

Hint: Use a dictionary to map roles to allowed resources. **

```
# Define roles and their allowed resources
role_permissions = {
   "customer": ["order_history"],
   "restaurant_owner": ["menu_edit", "order_history"],
   "admin": ["menu_edit", "order_history", "admin_dashboard"]
}
# Function to check access
def check_access(user_role, resource):
   if resource in role_permissions.get(user_role, []):
      return True
   else:
      return False
# Test cases
print("Customer - Order History:",
    check_access("customer", "order_history"))
print("Customer - Menu Edit:",
    check_access("customer", "menu_edit"))
print("Restaurant Owner - Menu Edit:",
    check_access("restaurant_owner", "menu_edit"))
print("Restaurant Owner - Admin Dashboard:",
    check_access("restaurant_owner", "admin_dashboard"))
print("Admin - Admin Dashboard:",
    check_access("admin", "admin_dashboard"))
```

3) Simulate a two-factor authentication flow like Paytm: after a user enters the correct password, generate a 6-digit OTP and display it in the console. Ask the user to input the OTP and display 'Login Successful' only if it matches.

```
import random
# Hardcoded login credentials
correct_username = "admin"
correct_password = "paytm123"
# Step 1: Enter username and password
username = input("Enter Username: ")
password = input("Enter Password: ")
if username == correct_username and password == correct_password:
    print("\nPassword Verified Successfully!")
    # Step 2: Generate 6-digit OTP
    otp = random.randint(100000, 999999)
    # Display OTP in console (simulation only)
    print("Your OTP is:", otp)
    # Step 3: Ask user to enter OTP
    user_otp = input("Enter 6-digit OTP: ")
    # Step 4: Verify OTP
    if user_otp == str(otp):
        print("\nLogin Successful")
    else:
        print("\nInvalid OTP. Login Failed.")
else:
    print("\nIncorrect Username or Password. Login Failed.")
```

**4) Given an array of user objects with properties: username, isPremium, and lastLogin, write a function that returns only those users who are premium and have logged in within the last 7 days.

Constraint: Assume lastLogin is a date string in 'YYYY-MM-DD' format, and today is '2024-06-10'.**

```
# Array of user objects
users = [
    {"username": "Hardik", "isPremium": True, "lastLogin": "2024-06-09"},
    {"username": "Rahul", "isPremium": False, "lastLogin": "2024-06-08"},
    {"username": "Amit", "isPremium": True, "lastLogin": "2024-06-01"},
    {"username": "Priya", "isPremium": True, "lastLogin": "2024-06-07"},
    {"username": "Neha", "isPremium": True, "lastLogin": "2024-06-10"}
]
# Today's date
today = datetime.strptime("2024-06-10", "%Y-%m-%d").date()
# Calculate date 7 days ago
seven_days_ago = today - timedelta(days=7)
# Filter premium users who logged in within the last 7 days
def filter_premium_users(users):
    return [
        user for user in users
        if user["isPremium"] and
        seven_days_ago <= datetime.strptime(
            user["lastLogin"], "%Y-%m-%d"
        ).date() <= today
    ]
# Display results
result = filter_premium_users(users)
print("Premium users logged in within the last 7 days:")
for user in result:
    print(user["username"])
```